

Mini Distributed File System (DFS)

A lightweight, fault-tolerant Distributed File System built from scratch in C++. This project demonstrates core distributed system concepts including centralized metadata management, file sharding, and high availability through data replication.



System Architecture

The system follows a client-server architecture with three main components communicating over custom TCP protocols:

1. **NameNode (The Brain):** Manages the file system namespace and metadata. It maps file names to the specific DataNodes where their sharded chunks are stored.
2. **DataNodes (The Storage):** Responsible for the actual storage and retrieval of data chunks on the local file system.
3. **Client:** Handles the heavy lifting of splitting files into chunks, negotiating with the NameNode, and uploading/downloading chunks to the appropriate DataNodes.



Key Features

- **Data Sharding (Chunking):** Large files are automatically split into 1KB chunks to distribute storage load and optimize network transfer.
- **High Availability (2x Replication):** Every chunk is mirrored across at least two independent DataNodes.
- **Automatic Failover:** If a primary DataNode crashes or becomes unreachable during a download, the Client seamlessly falls back to the replica node without losing data.
- **Custom Application Protocol:** Uses a lightweight **SAVE_CHUNK:** and **GET_CHUNK:** prefix protocol over TCP to ensure stream integrity.
- **Connection Resiliency:** Implements local backoff delays and explicit IPv4 binding to prevent socket exhaustion during massive file transfers.



Tech Stack

- **Language:** C++11 (or higher)
- **Networking:** POSIX Sockets (TCP/IP)
- **File I/O:** `<fstream>` binary reading/writing



How to Run the System

To test the system locally, you will need to open **four separate terminal instances**.

1. Start the NameNode

```
g++ namenode.cpp -o n
./n
2. Start the DataNodes
Open two new terminal tabs and run:
```

Bash

Terminal 2

```
g++ datanode.cpp -o d
./d 8081
```

Terminal 3

```
./d 8082
```

3. Run the Client

Create a `test` file and start the upload/download process:

Bash

Terminal 4

```
for i in {1..200}; do echo "Line $i: Testing Distributed Architecture" >>
hello.txt; done
```

```
g++ client.cpp -o c
```

```
./c
```

💥 Testing Fault Tolerance (The "Kill Test")

To verify the failover logic, simulate a server crash during the system's execution:

Start the NameNode and both DataNodes.

Run the Client to upload `hello.txt`.

Before the Client begins the download phase, kill one of the DataNodes (Press Ctrl+C in Terminal 2).

Watch the Client gracefully catch the connection failure, switch to the backup DataNode on port 8082, and successfully reassemble the complete file.

🔧 Cleanup

If ports get stuck in a `TIME_WAIT` state during testing, you can clear them using:

Bash

```
lsof -ti:9000,8081,8082 | xargs kill -9
```

```
rm -rf storage_8081 storage_8082 downloaded_hello.txt
```

🧠 Design Trade-Offs & Decisions

TCP vs. UDP: TCP (`SOCK_STREAM`) was chosen to guarantee packet ordering and absolute data integrity, which is non-negotiable for a file system.

Chunk Sizing: Tuned to 1KB chunks for local testing. In a production environment handling gigabytes of data, this would be increased to 64MB (similar to HDFS) to reduce NameNode memory overhead.

Stateless DataNodes: DataNodes do not talk to each other; they only respond to the Client. This simplifies the architecture and prevents network deadlocks during development.

Tips for your GitHub Repository:

1. ****Format the Code:**** Make sure your `.cpp`` files have consistent indentation before uploading.
2. ****Add a `.gitignore`:**